FARGOpy: A Python Package for Post-processing and Analyzing FARGO3D Hydrodynamical Simulations

Alejandro Murillo-Gonzalez[a,*], Jorge I. Zuluaga[a,*] and Matias Montesinos[b]

[a]SEAP/FACom, Instituto de Física - FCEN, Universidad de Antioquia, Calle 70 No. 52-21, Medellín 050026, Colombia
[b]Departamento de Física, Universidad Técnica Federico Santa María, Avenida España 1680, Valparaíso , Chile

ARTICLE INFO

Keywords:
Astronomy software (1855)
Hydrodynamical simulations (767)
Protoplanetary disks (1300)
Planet-disk interactions (1247)
Circumplanetary disks (235)
Gas accretion (575)
Open source software (1866)

ABSTRACT

We introduce FARGOpy, a Python package designed for the analysis and visualization of hydrodynamical simulations produced by the FARGO3D code. Its development responds to the need for a reproducible and physically consistent analysis pipeline for refined spherical grids, in which the curvilinear geometry and spatial non-uniformity hinder the use of generic visualization and analysis tools. The package allows for the manipulation of scalar and vector fields on refined spherical meshes, offering tools for multidimensional interpolation, flux calculation through arbitrary surfaces, and kinematic analysis in planetocentric frames. We describe the package's architecture, its workflow for data management, and validate its interpolation and integration schemes. FARGOpy provides a robust framework for post-processing planet-disk interaction simulations, facilitating the study of circumplanetary environments.

1. Introduction

FARGOpy is a post-processing package designed by the co-authors for the analysis and visualization of hydrodynamic simulations generated with FARGO3D. Its development responds to the need for a reproducible and physically consistent analysis pipeline for refined spherical grids, in which the curvilinear geometry and spatial non-uniformity hinder the use of generic visualization and analysis tools.

The package is published and publicly available through the official Python Package Index (PyPI) repository, which allows direct installation via standard Python package managers. FARGOpy is distributed under the *GNU Affero General Public License*, a free software license approved by the OSI, and is compatible with platform-independent operating systems. The development of the package has been carried out by Jorge I. Zuluaga (original conception, core classes and genera codign style, software architecture and documentation), Alejandro Murillo-Gonzalez (core analysis routines, interpolation, flux calculation, internal documentation) and MatÃŋas Montesinos (provider of the original scripts on which core methods were developed, scientific testing), and it falls within the areas of computational astronomy, fluid dynamics (CFD), and magnetohydrodynamics (MHD), with full support for Python 3.

The package source code is openly maintained in the official GitHub repository, where its internal structure, version history, and change tracking are documented. In addition to the core of the package, the repository includes a collection of interactive notebooks (Jupyter notebooks) that serve as usage examples, functional tests, and validation of different modules, including spatial interpolation, visualization of vector fields, flux calculation, and generation of animations. These notebooks constitute an integral part of the package verification process and facilitate the reproducibility of the analyses.

FARGOpy operates exclusively on the data stored in the simulation *outputs*, without intervening in the numerical solution of the hydrodynamic equations or modifying the temporal evolution of the system. In this way, the package acts as an analysis layer independent of the solver, ensuring that the physical diagnostics presented in this work are derived directly from the hydrodynamic fields produced by FARGO3D.

The design of FARGOpy is guided by the following fundamental principles: (i) code readeability, (ii) physical consistency in all operations performed on the data, including the explicit treatment of geometry and units; (iii) traceability of every stage of the analysis, avoiding implicit or ambiguous transformations; and (iv) full potential to be used in teaching or training environments.

In this context, FARGOpy does not aim to replace general-purpose interactive visualization tools, but rather to provide a specialized environment for the quantitative analysis of flows, structures, and physical diagnostics relevant to the study of circumplanetary disks and their immediate surroundings, integrating numerical control, visualization, and validation within a unified framework.

2. Workflow and data management

The simulations performed with FARGO3D produce their results as independent binary files (.dat) stored snapshot by snapshot. Each file contains a data cube corresponding to a hydrodynamic field defined on the solverâĂŹs spherical mesh, discretized in coordinates (r, θ, ϕ). Scalar fields (for example, density and internal energy) and vector fields (velocity field components) are stored separately, preserving the native discretizations and the spatial refinement applied during the simulation.

*Corresponding author
alejandro.murillo1@udea.edu.co (A. Murillo-Gonzalez); jorge.zuluaga@udea.edu.co (J.I. Zuluaga); matias.montesinosa@usm.cl (M. Montesinos)
ORCID(s): 0009-0005-4433-2474 (A. Murillo-Gonzalez); 0000-0002-6140-3116 (J.I. Zuluaga); 0000-0001-9789-5098 (M. Montesinos)

In addition to the hydrodynamic fields, FARGO3D stores global information associated with the simulation, including the physical and numerical parameters defined in the setup, such as the stellar mass, the fundamental length, mass, and time scales, and the relevant hydrodynamic parameters. This information defines the simulated physical system and is required for a consistent interpretation of the data.

In FARGOpy, data loading and organization are handled through the Simulation module (and the corresponding class), which centralizes access to the global simulation parameters, physical units, and the orbital information of the embedded bodies. This module makes it possible to explicitly reconstruct the complete physical context of each simulation, ensuring that both the user and the analysis pipeline have direct access to all the parameters required for subsequent calculations.

Hydrodynamic fields are managed by the Fields module (and several associated classes), which is responsible for reading the original binary files without modifying their numerical values and for reorganizing the geometric information associated with each cell. During this process, the data are internally transformed into a tabular representation using Pandas DataFrames or Numpy Arrays, in which each volume element is explicitly associated with its coordinates (either Cartesian (x, y, z) or any of the other systems available in FARGO3D), as well as with the corresponding scalar and vector field values. This representation facilitates subsequent operations such as spatial filtering, multidimensional interpolation, geometric tessellation, and the computation of physical integrals. The data are organized independently for each snapshot, preserving the original temporal information of the simulation and avoiding the introduction of additional assumptions about the dynamical evolution of the system.

In a complementary way, FARGOpy explicitly loads the orbital information of the planet or planets stored in the outputs of FARGO3D, including its mass, position, and velocity at each snapshot. This snapshot-by-snapshot tracking makes it possible to define regions centered on the planets and to compute quantities that depend on its orbital dynamics, such as the Hill radius and the delimitation of circumplanetary regions, which may vary over time.

Based on the fundamental scales defined in the simulation, the data can be automatically transformed into systems of physical units consistent with the simulation, either in the CGS system or in the MKS system, depending on the userâĂŹs choice. This conversion is applied uniformly to all fields, parameters, and diagnostics, ensuring an unambiguous physical interpretation and allowing direct comparison between simulations configured with different scale parameters.

Taken together, the coordinated loading of the hydrodynamic fields, the global simulation parameters, and the orbital information of the planets establishes a clear separation between the original data produced by the solver and its internal representation in FARGOpy. This structure forms the basis of the analysis pipeline used in this work, ensuring physical consistency, traceability, and reproducibility at all subsequent stages.

In order to illustrate FARGOpy typical code and the workflow used to process FARGO3D output, we include below a typical code snippet.

```
# Tested in version 1.0.5 of FARGOpy
import fargopy as fp
import numpy as np
import matplotlib.pyplot as plt

sim_path = fp.Simulation.download_precomputed('pds70iso')
sim = fp.Simulation(output_dir=sim_path)
sim.units("CGS")

dens = sim.load_field(
    fields='gasdens',
    slice='r=[0.8,1.2],phi=[-25 deg,25 deg],theta=1.56',
    snapshot=200,
    interpolate = False
)

plt.pcolormesh(
     dens.mesh.x,
     dens.mesh.y,
     np.log10(dens.data)
)
```

For this example, we first downloaded one of the precomputed simulations FARGO3D provided with the package. These precomputed simulations are intended to facilitate the learning process without the burden of waiting for a partial or full simulation of FARGO3D (for a list of precomputed simulations, use the function fp.Simulation.list_precomputed()). Subsequently, a slice of the gas density field is loaded at a specific time (snapshot). The slice corresponds to a horizontal section in the middle of the disk ($\theta \simeq \pi/2$). The field object dens comes with the coordinate mesh data.mesh, which could be used in calculations or, as shown in the code, to plot the field.

Anyone familiar with FARGO3D can attest that the same procedure, programmed without the help of FARGOpy, takes, depending on experience, from several tens of lines to more than a hundred lines of code.

3. Multidimensional interpolation and resampling on irregular meshes

The hydrodynamical simulations produced with FARGO3D are defined on a refined, non-uniform spherical mesh, optimized for the numerical integration of the hydrodynamical equations in the presence of strong gradients and localized regions of interest. Although this discretization is suitable for the time evolution of the system, it imposes practical limitations on the subsequent analysis of the data.

Thus, for instance, in a spherical geometry and variable spatial resolution, it is difficult to directly construct local Cartesian slices, which are necessary to characterize three-dimensional structures in the vicinity of the planets, such as circumplanetary disks, accretion flows, or shock regions, or in other regions of interest in the simulations. Similarly, obtaining physically comparable projections between

different snapshots or simulations requires evaluating the hydrodynamical fields on regular grids that, in general, do not coincide with the cell centers of the native mesh.

On the other hand, a homogeneous comparison between models with different effective resolutions or distinct spatial refinement demands controlled resampling of the data. Working directly on the original mesh can introduce discretization-dependent biases, hindering the physical interpretation of differences between simulations that stem from different parameters rather than numerical effects.

Additionally, many of the FARGO3D analysis operations –such as the definition of arbitrary geometric regions, the calculation of fluxes through surfaces, or the evaluation of fields in reference frames centered on the planet– require evaluating physical quantities at positions that do not belong to the original mesh. In this context, multidimensional interpolation is a necessary tool to connect the discrete data produced by the solver with continuously defined physical diagnostics, while preserving the geometric coherence of the simulated domain.

For solving these issues, FARGOpy includes an interpolation module specifically designed to work with irregular spherical meshes, allowing scalar and vector fields to be mapped to regular one-, two-, or three-dimensional representations in a consistent and controlled manner.

In the package, interpolation is conceived as a resampling operation that makes it possible to evaluate hydrodynamical fields, discretely defined on the native FARGO3D mesh, at arbitrary positions in space. The original fields are defined on a refined, non-uniform spherical mesh, which requires an explicit treatment of the geometry before applying any interpolation scheme. For this purpose, the coordinates of the cell centers are explicitly reconstructed in Cartesian coordinates, allowing the interpolation to be written as a weighted combination of neighboring discrete values,

$$f(\mathbf{x}) = \sum_i w_i(\mathbf{x})\, f_i, \tag{1}$$

where f_i denotes the value of the field in the i-th cell of the original mesh and w_i are the weights associated with the chosen interpolation scheme. This formulation is independent of the dimensionality of the problem and is applied consistently to one-dimensional slices, two-dimensional slices, or full three-dimensional volumes.

Interpolation is explicitly restricted to the physical domain covered by the simulation. FARGOpy implements geometric masks that ensure that the evaluation of the fields is carried out only within the radial and angular boundaries defined by the original mesh, avoiding uncontrolled extrapolations outside the simulated region. This control is especially relevant for Cartesian cuts and local projections, where the geometry of the native mesh does not coincide with the evaluation grid.

When multiple snapshots are used, spatial interpolation is performed independently on each of them. Subsequently, if the user so requires, the interpolated values are combined by means of linear interpolation in time, allowing hydrodynamic fields to be evaluated at intermediate instants without introducing additional assumptions about the dynamical evolution of the system.

3.1. Implemented interpolation schemes

FARGOpy offers different spatial interpolation backends for resampling fields defined on irregular grids. The main backend is based on the `griddata` function from `SciPy`, which allows interpolating data defined on arbitrary sets of points via an implicit triangulation of the domain. This backend supports linear, nearest-neighbor, and cubic interpolation. Additionally, the package includes alternative interpolators based on radial basis functions (RBF), inverse distance weighting (IDW) schemes, and methods of the `LinearNDInterpolator` type.

In this work, the default scheme adopted is the linear interpolation implemented in `griddata`. Linear interpolation preserves the local structure of hydrodynamic fields and maintains the uniformity of the interpolated variables, reducing the appearance of non-physical oscillations in regions characterized by strong gradients, highly compressible flows, or shocks. These properties are particularly relevant in three-dimensional simulations with variable spatial refinement.

Higher-order methods, such as cubic interpolation or interpolators based on radial basis functions, can induce excessive smoothing or generate numerical artifacts when applied to non-uniform grids. For this reason, such schemes are not used in the quantitative diagnostics presented in this work. Their use is explicitly allowed within FARGOpy for exploratory analysis or visualization purposes, where the visual continuity of the field takes precedence over the strict preservation of its local structure.

In the example shown in the code below, we demonstrate how representations and calculations can be performed on interpolated meshes (resampling), even in the more complex case of velocity fields. In the code, we have omitted the lines corresponding to downloading the simulation, reading the configuration files, and defining the units, as they are exactly the same as in the code shown previously.

```
fields = sim.load_field(
    fields=['gasv','gasdens'],
    slice="r=[0.6,1.4], phi=0",
    snapshot=200
)

# Create the mesh grid for interpolation
X, Z = np.meshgrid(
    np.linspace(fields.var1_mesh[0].min(),
                fields.var1_mesh[0].max(), 120),
    np.linspace(fields.var3_mesh[0].min(),
                fields.var3_mesh[0].max(), 120),
    indexing='ij'
)

# Interpolate
gasdens_interp = fields.evaluate(
    field='gasdens', time=0, var1=X, var3=Z
)
velocity_interp = fields.evaluate(
```

```
    field='gasv', time=0, var1=X, var3=Z
)

# Colormesh of density
axs.pcolormesh(
    X, Z, np.log10(gasdens_interp),
    cmap='Spectral_r'
)
# Streamlines of velocity
axs.streamplot(
    X.T, Z.T,
    velocity_interp[0].T, velocity_interp[2].T,
    linewidth=0.7, density=3.0
)
```

The gain in this case, in terms of both breadth and clarity of the analysis code, is already significant. The same procedure as in the previous code, carried out with code snippets of your own code or othersâĂŹ of people who work with FARGO3D can easily be extended to a couple of hundred lines of code. Ignoring stylistic issues (eg, the vertical hanging indent used for fitting the horizontal size of the lines of code to the format of this manuscript), in FARGOpy this procedure takes only 6 lines of code in total and is far more readable and maintainable.

3.2. Validation of the two-dimensional interpolation of the density field

In order to assess the fidelity of the two-dimensional interpolation scheme used in FARGOpy, a validation was carried out specifically aimed at quantifying the error introduced during the resampling process of hydrodynamical fields defined on irregular meshes.

Unlike approaches based on spatial cross-validation, whose goal is to evaluate the predictive capability of the interpolator in unsampled regions, the purpose here is to verify that the interpolation preserves the physical information of the field when it is used exclusively as a *regridding* operator. This is the relevant scenario for the analyses presented in this work, where interpolation is employed to map simulated fields onto regular grids to facilitate subsequent calculations, and not to extrapolate information beyond the resolved domain.

For our test, the data analyzed correspond to three-dimensional hydrodynamical simulations run with FARGO3D of the CPD of PDS-70c. The interpolation evaluated corresponds to the fields in a meridional slice defined by $\phi = 0$ and a radial interval $r \in [0.8, 1.2]$ centered on the planetary orbit. This region encompasses the immediate circumplanetary environment, characterized by strong spatial density gradients associated with both the planet's gravitational potential and the vertical stratification of the disk. Since the simulations resolve only the upper half-disk, the analysis domain is restricted to $z \geq 0$. In Figure 1, the result of this validation test can be visually examined.

The validation is based on a round-trip test, constructed as follows. First, the original density field ρ_{true} (panel a in Figure 1), defined on the irregular simulation mesh, is linearly interpolated onto a high-resolution regular Cartesian grid (panel b in Figure 1) using interpolation based on a Delaunay triangulation, implemented with the griddata function from SciPy (Virtanen et al., 2020a). Subsequently, the field interpolated on the regular grid is evaluated again at the original positions of the irregular mesh, yielding a reconstructed field ρ_{back} (panel c in Figure 1). The point-by-point comparison between ρ_{true} and ρ_{back} makes it possible to isolate the error introduced solely by the interpolation operator, without involving changes in the effective resolution or additional physical processes.

The point-by-point consistency between both fields is quantitatively assessed in Figure 2, which presents the scatter plot between ρ_{back} and ρ_{true} in logarithmic scale, the distribution of $|\Delta \log_{10} \rho|$, and the spatial map of the absolute relative error. The scatter plot closely follows the identity line across the entire dynamic range of the field, which spans more than six orders of magnitude in density, without showing any systematic offsets.

From a quantitative standpoint, the error introduced by the interpolation is small throughout the considered domain. The median of $|\Delta \log_{10} \rho|$ is 1.3×10^{-4}, corresponding to a multiplicative factor of 1.0003, while 68% of the points have errors below 0.06%. 95% of the points remain below 0.34%, and even at the 99th percentile the error corresponds to a multiplicative factor of only 1.026. The mean relative bias is on the order of 10^{-3}, consistent with the absence of any appreciable systematic shift in field reconstruction.

These results show that the linear interpolation scheme used in FARGOpy preserves the density field with high fidelity when used as a resampling operator. The error introduced by interpolation is sub-percent over the vast majority of the domain and remains confined to regions of strong spatial variation, as expected for a linear scheme applied to hydrodynamical fields with sharp gradients. Since these errors are significantly smaller than the physically relevant variations of the system and do not introduce systematic biases, we conclude that interpolation is not a limiting factor for the scientific analyzes presented in this work.

4. Surface tessellation

FARGOpy implements analytical spherical, cylindrical, and planar surfaces, defined in Cartesian coordinates and parameterized by their center, extent, and orientation. These surfaces are used as integration domains for computing fluxes and accretion rates in regions of interest, such as the circumplanetary environment.

Spherical surfaces are discretized by recursively subdividing a regular icosahedron, generating an approximately uniform triangular tessellation over the sphere. This approach allows explicit control over the angular resolution of the integration surface and ensures homogeneous coverage of the domain. Each triangle in the tessellation is characterized by its geometric center, its area, and an outward-pointing unit normal vector.

Cylindrical surfaces are discretized by explicitly separating the contributions from the top and bottom caps and from

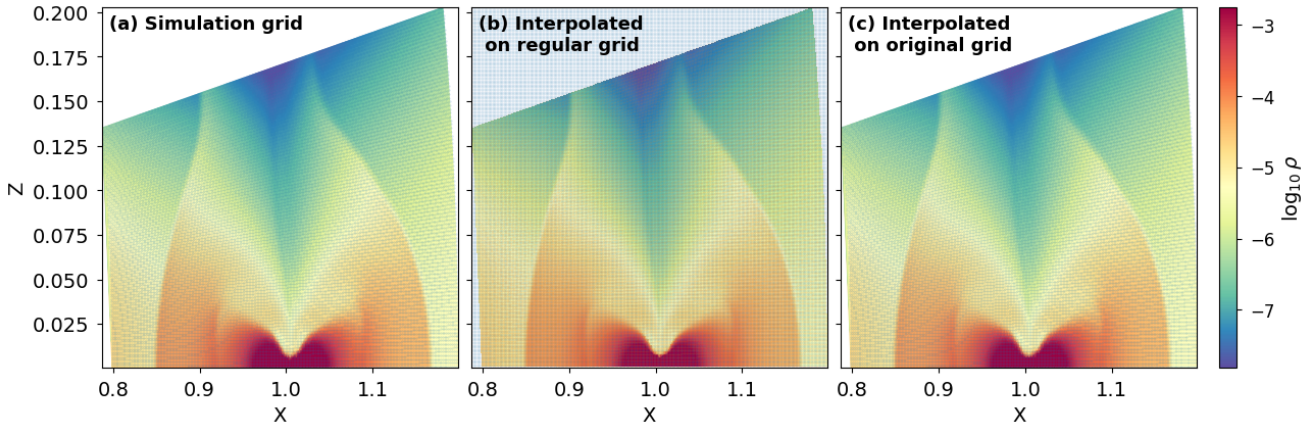


Figure 1: Visual comparison of the density field in the meridional slice under consideration. *Left:* original density field defined on the irregular simulation mesh. *Center:* field interpolated onto a high-resolution regular Cartesian grid. *Right:* field reconstructed on the original mesh after the round-trip process. All three panels are shown with the same normalization in $\log_{10}\rho$. The interpolation preserves the global morphology of the field, including the vertical stratification of the disk and the structures associated with the planetary gravitational well, without introducing spatial artifacts.

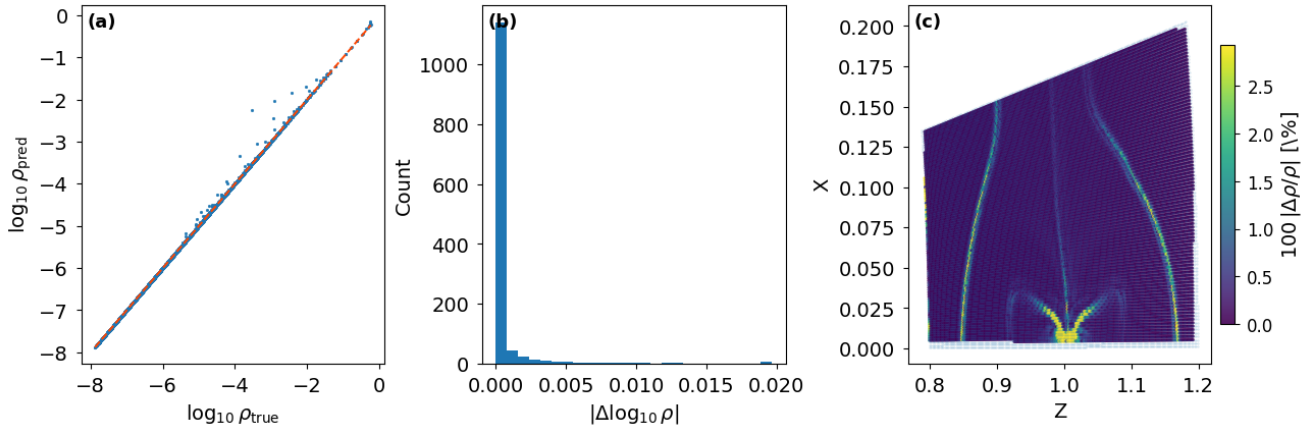


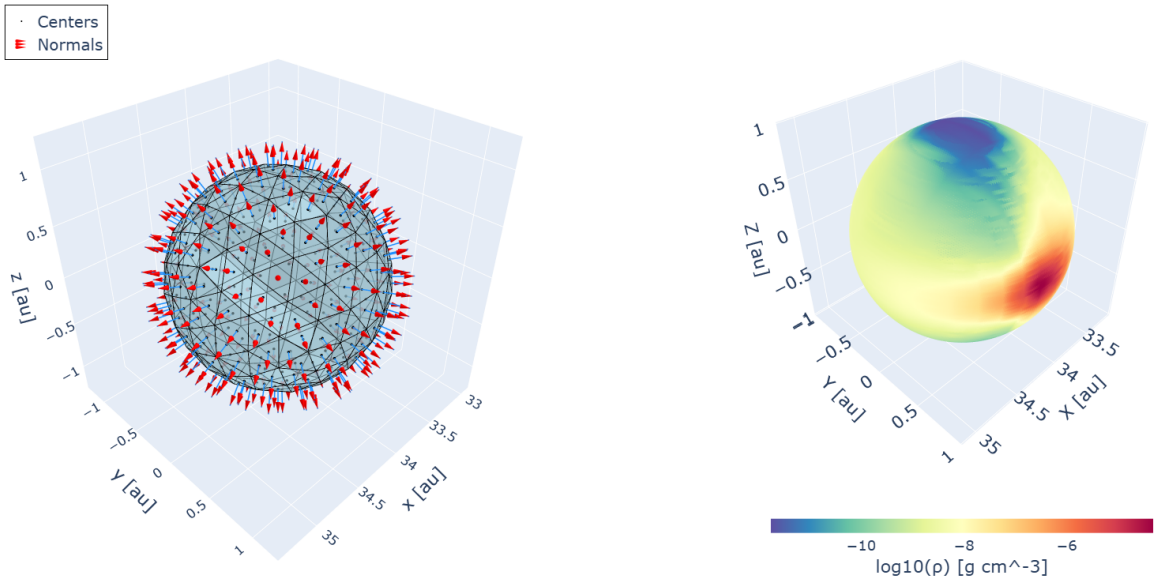
Figure 2: Quantitative diagnostics of the roundâĂŚtrip test applied to the density field. *Left:* scatter plot between the reconstructed field $\rho_{\rm back}$ and the original field $\rho_{\rm true}$ in logarithmic scale; the dashed line indicates the identity relation. *Center:* distribution of $|\Delta \log_{10}\rho|$, which shows a strong concentration at small values. *Right:* spatial map of the absolute relative error $100\,|\rho_{\rm back}-\rho_{\rm true}|/\rho_{\rm true}$, expressed as a percentage. The largest discrepancies are concentrated in regions with sharp spatial gradients, such as the vicinity of the midâĂŚplane and the structures associated with shocks. Although these discrepancies are relatively larger compared to the rest of the domain, their absolute magnitude is small and they do not give rise to extended regions.

the lateral surface. Each of these regions is divided into rectangular panels, defined so as to preserve the effective area and the correct orientation of the normal vectors. Similarly, planar surfaces are discretized into rectangular cells aligned with the Cartesian axes, making it possible to define local cuts of finite extent.

In all cases, the tessellation yields a discrete set of area elements $\{A_k\}$, each associated with a center $\mathbf{x}_k$ and a normal vector $\hat{\mathbf{n}}_k$, forming the basis for the numerical evaluation of the physical integrals. In Figure 3 we illustrate the tesselation scheme used by FARGOpy for calculating fields on arbitrary surfaces.

4.1. Computation of enclosed mass and mass flux in FARGOpy

The FARGOpy package implements a set of geometric and numerical routines designed to consistently quantify the gas mass associated with the circumplanetary environment and the mass fluxes that cross control surfaces centered on the planet. These tools are designed to operate directly on the three-dimensional outputs of FARGO3D in spherical coordinates, respecting both the geometry of the native mesh and the possible orbital migration of the planet throughout the simulation. The main goal is to define physically well-motivated volumes and surfaces, typically scaled with the



(a) (b)

Figure 3: Construction and application of a tessellated spherical surface for sampling three-dimensional fields in the vicinity of the planet. Panel (a) shows the tessellation of the spherical surface together with the centers of th surface elements and the outward normal vectors, used to define the local orientation of each cell. Panel (b) shows the gas density field interpolated and projected onto the same spherical surface, color-coded according to $\log_{10}(\rho)$, illustrating the spatial variation of the density in the circumplanetary environment.

Hill radius, that move with the planet and allow one to evaluate gas accretion and recirculation processes.

The calculation of the mass flux is based on the continuous definition

$$\Phi_M(t) = \int_{S(t)} \rho(\mathbf{x}, t)\, \mathbf{v}_{\rm rel}(\mathbf{x}, t) \cdot \hat{\mathbf{n}}(\mathbf{x}, t)\, dA, \tag{2}$$

where $S(t)$ is a closed surface centered on the planet, ρ is the gas volume density, $\hat{\mathbf{n}}$ is the outward normal to the surface, and $\mathbf{v}_{\rm rel}$ is the gas velocity in the chosen frame. In circumplanetary applications it is natural to adopt the non-inertial frame comoving with the planet, so that $\mathbf{v}_{\rm rel} = \mathbf{v} - \mathbf{v}_p$, with $\mathbf{v}_p(t)$ the instantaneous orbital velocity of the planet. This choice ensures that the measured flux reflects genuine accretion or ejection of gas with respect to the planet, and not a kinematic artefact associated with its translation. It is worth noting that this strategy can be applied equally well to compute the flux of any other dynamical quantity of interest, such as the angular momentum transported across a control surface.

Numerically, the surface $S(t)$ is discretized by means of an explicit tessellation. In the spherical case, one starts from a regular icosahedron whose triangular faces are recursively subdivided until the desired angular resolution is reached. Each triangle defines an elemental patch characterized by its area A_k, its geometric center $\mathbf{x}_k$, and a unit normal vector $\hat{\mathbf{n}}_k$ oriented outward. The total flux is then approximated as

$$\Phi_M(t) \simeq \sum_{k=1}^{N} \rho_k(t)\, \left[\mathbf{v}_{{\rm rel},k}(t) \cdot \hat{\mathbf{n}}_k(t)\right]\, A_k(t), \tag{3}$$

where ρ_k and $\mathbf{v}_{{\rm rel},k}$ are evaluated at the center of each patch by linear interpolation of the hydrodynamic fields from the original FARGO3D grid. This procedure explicitly preserves the geometric orientation of the surface and allows one to integrate fluxes over complete or truncated surfaces, for example hemispheres defined by $z \geq z_{\rm cut}$, which is necessary in simulations that only resolve the upper half-disk.

A key aspect of the method is the explicit tracking of the planet throughout the simulation. In each `snapshot`, the instantaneous position of the planet $\mathbf{x}_p(t)$ is extracted directly from the FARGO3D output files, and the center of the control surface is redefined accordingly as $\mathbf{x}_c(t) = \mathbf{x}_p(t)$. This procedure ensures that all volumetric and surface integrals are evaluated in a planet-centered reference frame, allowing a consistent measurement of the gas properties within the Hill sphere despite planetary migration or numerical displacement. By redefining the control volume at every output, the method isolates the true circumplanetary flow and avoids spurious contributions from the background circumstellar disk.

Since the tessellation is defined in absolute Cartesian coordinates, this shift requires a complete reconstruction of the surface at each instant. In this way, the integration surface faithfully follows the orbital migration of the planet and remains centered on the physical object of interest. Additionally, when one wishes to work in units of the Hill radius, the size of the surface is dynamically rescaled as $R(t) = f\, R_{\rm Hill}(t)$, where f is a constant factor set at the beginning of the analysis. This ensures that the integrated region always represents the same fraction of the gravitationally bound

environment of the planet. The total mass of the enclosed gas is computed independently by direct integration over the volume delimited by the control surface. The continuous definition is

$$M(t) = \int_{V(t)} \rho(\mathbf{x}, t)\, dV, \tag{4}$$

where $V(t)$ is the volume associated with the surface $S(t)$. Since FARGO3D uses spherical coordinates (r, θ, ϕ), the volume element is given by $dV = r^2 \sin\theta\, dr\, d\theta\, d\phi$. In practice, this integral is evaluated as a discrete sum over the cells of the native mesh:

$$M(t) \simeq \sum_{i,j,k \in \mathcal{I}(t)} \rho_{ijk}(t)\, r_i^2 \sin\theta_j\, \Delta r_i\, \Delta\theta_j\, \Delta\phi_k, \tag{5}$$

where $\mathcal{I}(t)$ denotes the set of cells whose geometric centers lie within the volume of interest. To construct this volumetric mask, the spherical coordinates of each cell are transformed to Cartesian coordinates and recentered with respect to the position of the planet, $\mathbf{x}' = \mathbf{x} - \mathbf{x}_p(t)$, so that the geometrical criterion for a sphere of radius $R(t)$ is simply $|\mathbf{x}'| \leq R(t)$, with additional conditions in z when truncated volumes are considered. This approach makes it possible to integrate the mass directly over the hydrodynamic cells, incorporating the geometric Jacobian of the mesh exactly and avoiding errors associated with volumetric interpolations.

The combination of both techniques, surface flux and enclosed mass, allows a consistent evaluation of the volume-integrated continuity equation,

$$\frac{dM}{dt} = -\Phi_M, \tag{6}$$

in the absence of explicit source terms. Within this framework, any residual discrepancy between the temporal variation of the mass and the integrated flux can be attributed in a controlled manner to numerical effects such as spatial resolution, time discretization of the snapshots, or the geometric quantization imposed by the mesh. This approach provides a solid foundation for the quantitative analysis of gas accretion and circulation in circumplanetary disks within FARGOpy.

The following code snippet shows the calculation of the gas mass enclosed within the planet's Hill radius, by integrating the density field over a spherical surface centered on the instantaneous position of a Jovian planet in a circumstellar disk.

```
simpath = fp.Simulation.download_precomputed('p3disoj')
sim = fp.Simulation(output_dir=simpath)

# Load planet properties
planet = sim.load_planets(snapshot=10)[0]

# Define spherical surface at the Hill radius
sphere = fp.Surface(
    type="sphere",
    center=(planet.pos.x, planet.pos.y, planet.pos.z),
    radius=planet.hill_radius,
    subdivisions=6,
    z_cut=0.0
)

# Compute enclosed gas mass
mass = sphere.total_mass(
    sim,
    snapshot=[0,10],
    follow_planet=True
)

# Compute accreation rate
flux = sphere.mass_flux(
    sim=sim,
    snapshot=[0,10],
    follow_planet=True
)
```

Note that it is not necessary to load the data from the density field to perform this calculation: FARGOpy automatically reads the required data (the density in this case) and uses it to compute the total mass. The power of FARGOpy is demonstrated here not only in organizing the output data from FARGO3D but also in carrying out truly complex physical calculations with just a few lines of code.

4.2. Enclosed mass and mass flux

Given a tessellated surface, FARGOpy can compute (i) enclosed mass proxies (by combining surface sampling with appropriate geometric factors and/or volume elements, depending on the diagnostic) and (ii) mass flux through the surface using $\dot{M} = \int \rho\, \mathbf{v} \cdot d\mathbf{A}$. These quantities are used in the main text to characterize accretion and flow morphology around the planet.

4.3. Validation of 3D interpolation on spherical surfaces

The validation of the three-dimensional interpolation implemented in FARGOpy aims to demonstrate that it is possible to reconstruct observables defined on spherical surfaces from the original discrete data of 3D hydrodynamical simulations carried out with FARGO3D. This appendix specifically evaluates the reconstruction of the gas density field on a tessellated sphere centered on the planet, using the high-resolution fiducial simulation fid2 This test provides a strict internal validation of the interpolation method employed, designed to quantify its stability, convergence, and possible limitations in the absence of an analytical reference solution.

The high-resolution fid2 simulation provides a dense set of samples of the density field on a three-dimensional curvilinear mesh. For this validation, a local volume around the planet is extracted, defined as a Hill semisphere consistent with the half-disk geometry of the simulation. In the volume considered, there are approximately 1.28×10^4 effective hydrodynamical cells, which constitute the full set of candidate points for training the interpolation. From this set, nested subsets of increasing size N are constructed, such that each subset strictly contains the previous one. This procedure ensures that the differences observed between consecutive levels reflect only the effect of increasing the sampling density and not random fluctuations due to independent selections.

The evaluation surface is defined as a sphere with fixed radius, expressed as a fraction of the planet's Hill radius,

centered at its instantaneous position. This sphere is tessellated by successive subdivisions of a regular icosahedron, generating a set of triangles that are nearly isotropic in area. The surface observable is evaluated exclusively at the geometric centers of these triangles, and each value is associated with a well-defined area. This construction is necessary because it allows the spherical map to be treated as a controlled discretization of a continuous surface, rather than as an arbitrary collection of points.

Because the simulation is of half-disk type, the region corresponding to negative latitudes does not represent independent physical information, but rather a geometric extension by reflection. Consequently, all quantitative metrics in this validation are computed only over the physical hemisphere, defined by latitude ≥ 0. This restriction prevents introducing a false sense of convergence associated with imposed symmetries and ensures that the results reflect exclusively the behavior of the interpolation in the physically simulated domain.

The three-dimensional interpolation is performed in Cartesian space (x, y, z) using `scipy.interpolate.griddata` with the `linear` method. This scheme corresponds to piecewise linear interpolation based on the Delaunay tetrahedralization of the training point set. For evaluation points that fall outside the convex hull of the training set, the linear interpolation is not defined and returns non-numerical values. In these cases, a strictly local fallback scheme based on nearest-neighbor assignment is applied. This fallback does not modify the solution in regions where the linear interpolation is valid and is used exclusively to avoid undefined values, while also allowing explicit identification of the regions where the reconstruction depends on geometric extrapolation. Given that there is no exact solution for the density at the centers of the sphere, the validation is based on an internal consistency criterion between successive reconstructions. For each triangle center, the absolute fractional change between two consecutive training levels is defined as

$$\frac{|\Delta\rho|}{|\rho|} = \frac{\left|\rho_{N_{k+1}} - \rho_{N_k}\right|}{\left|\rho_{N_{k+1}}\right|} \tag{7}$$

where ρ_{N_k} is the interpolated density using N_k training points. This definition provides a dimensionless measure that can be directly interpreted as a relative variation, suitable for a strictly positive field such as the density. All statistics are computed weighting by the area of each triangle in the tessellation, so that the percentiles correctly reflect the contribution of each region to the total area of the physical hemisphere. This step is important, since the observables of interest in the rest of the work (spherical maps, surface averages, integrals over shells) depend on the surface measure and not on the number of sampling points.

The results of this validation are presented in Figure 4. Panel (a) shows the evolution of the area-weighted percentiles ($P50$, $P90$, and $P95$) of the fractional change $|\Delta\rho|/|\rho|$ as a function of the number of training points N. A systematic decrease of all percentiles is observed as N increases, followed by a clear tendency to saturation for $N \gtrsim 2 \times 10^4$. In particular, the median reaches values of order 10^{-3} in the highest-sampling regime, indicating that for at least half of the area of the physical hemisphere the density reconstructed on the sphere is practically invariant under any further refinement of the training set. The high percentiles, which are sensitive to the most problematic regions of the map, stabilize at values of a few percent, implying that the residual discrepancies are confined to a limited fraction of the surface area.

The comparison between the curves corresponding to pure linear interpolation and those that include the nearest-neighbor fallback is particularly diagnostic. In the high-N regime, both curves are practically identical, demonstrating that most of the evaluation surface lies within the convex hull of the training set. Therefore, the reconstructed map is controlled by the linear interpolation rather than by the fallback scheme. This result rules out the main failure mode of sparse interpolation in three dimensions, namely the dominance of uncontrolled extrapolations in insufficiently sampled regions.

Panel (b) of Figure 4 shows the distribution of the fractional change $|\Delta\rho|/|\rho|$ for the final pair of training levels. The distribution exhibits a very sharp peak near zero and a low-amplitude tail towards higher values. This morphology indicates that most of the surface area experiences very small variations between the two most refined levels, while the large values of the fractional change correspond to a reduced subset of triangles. In the context of a planetâĂŞdisc system, this behavior is expected, since the density field is not globally smooth, but instead contains localized structures and steep gradients associated with shocks, spirals, accretion flows, and shear in the circumplanetary environment. In these regions, the accuracy of any first-order interpolation scheme is limited by the local sampling resolution and by the hydrodynamic discretization itself.

Panel (c) shows the map of the fractional change $|\Delta\rho|/|\rho|$ in longitude-latitude coordinates for the physical hemisphere. The residual discrepancies are arranged in spatially localized patches, rather than being uniformly distributed. This pattern confirms that the residual error does not correspond to global numerical noise or to a methodological instability, but instead to specific physical regions characterized by strong field gradients. The overplotted markers indicate the points where linear interpolation is not defined at the highest N level; their areal coverage is small, consistent with the reduced separation between the linear curves and with the fallback observed in panel (a) of Figure 4.

This validation shows that the three-dimensional linear interpolation used in `FARGOpy` makes it possible to reliably reconstruct the gas density on tessellated spherical surfaces from the original simulation data, provided that the training volume adequately covers the evaluation surface in the physical domain. The systematic convergence of the area-weighted percentiles, the limited influence of the fallback

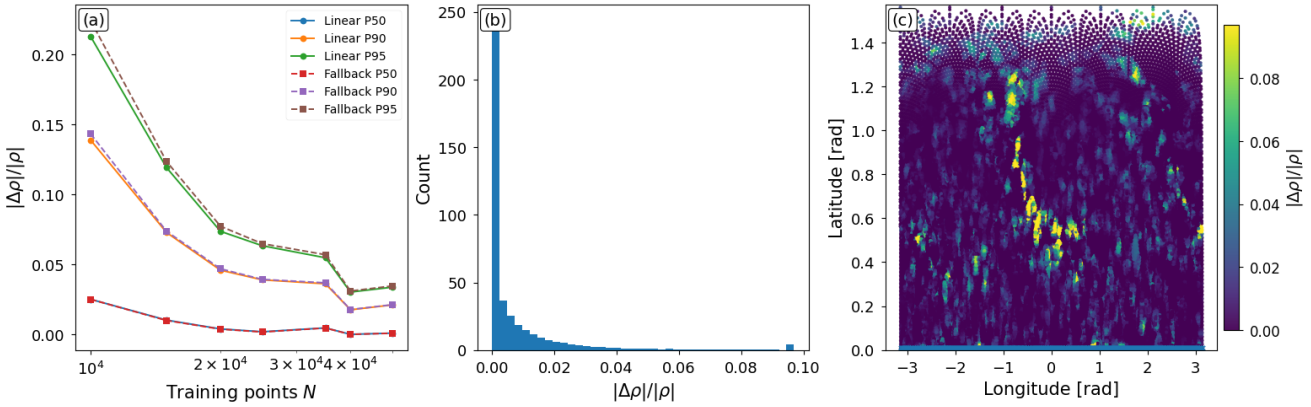


Figure 4: Validation of the 3D interpolation of the gas density field on a tessellated spherical surface for the high-resolution $4M_{Jup}$ simulation. All metrics are computed only on the physical hemisphere (latitude ≥ 0). (a) Area-weighted percentiles of the fractional change $|\Delta\rho|/|\rho|$ between consecutive training levels as a function of the number of points N, comparing linear interpolation and linear interpolation with nearest-neighbor fallback. (b) Distribution of $|\Delta\rho|/|\rho|$ for the final pair of levels. (c) LongitudeâĂŞlatitude map of the fractional change for the final pair; the markers indicate locations where linear interpolation is not defined at the highest-N level.

scheme, and the spatial localization of the residual discrepancies indicate that the resulting spherical maps are not artifacts of sparse sampling, but stable projections of the underlying hydrodynamical field. Consequently, the use of FARGOpy together with `griddata` in linear mode is justified for the construction of spherical maps and surface diagnostics throughout this work, with the usual caveat that strictly pointwise interpretations in regions of extreme gradients should be treated with caution, as is appropriate for any first-order interpolation scheme.

4.4. Graphical Interface

FARGOpy includes an optional interactive graphical interface (the package is in fact originally designed to be used in scripts and notebooks as support for research work). It is designed to facilitate the exploration, diagnosis, and systematic visualization of data generated by hydrodynamic simulations with FARGO3D. This interface acts as a visual control and analysis layer over the simulation outputs, allowing efficient inspection of the spatial and temporal structure of the hydrodynamic fields without the need to resort to case-specific scripts.

The interface is built on the `PyQt5` library, natively integrating `matplotlib` visualization routines through an embedded canvas and standard interactive tools (zoom, panning, and region selection). Communication with the simulation data is handled through the FARGOpy API, ensuring consistent access to the physical fields, computational domain information, and planetary orbital parameters, while preserving the units and scales defined in the original simulation.

From a functional perspective, the interface allows the dynamic selection of the output directory of a simulation, navigation between temporal snapshots, the definition of arbitrary two-dimensional cuts in spherical coordinates via ranges in r, θ, and ϕ, and switching between different types of physical maps, including density, internal energy, and components of the velocity field. The fields can be visualized both on the original mesh and on an interpolated regular grid, which is particularly useful for morphological analysis and for comparison between simulations with different resolutions.

The tool also offers detailed control over visualization parameters, including the selection of colormaps, the use of fixed color bars or ones normalized to reference snapshots, and the manual setting of dynamic limits. For kinematic analysis, it is possible to overlay flow streamlines, color-coded by velocity magnitude, as well as to adjust their density and arrow scale. The interface also includes visualization of the planet's Hill radius and the option to reflect spatial cuts with respect to defined axes, which facilitates the identification of asymmetries and the assessment of numerical symmetries in the flow. A relevant aspect of the interface is its explicit handling of physical units. The user can interactively inspect and modify the length and mass units of the simulation, as well as choose the scale in which the spatial axes are represented (simulation units, CGS, MKS, or astronomical units), ensuring consistency between the numerical values displayed and their physical interpretation.

Finally, the interface allows for the automated generation of time sequences and videos from the simulation snapshots, with control over the time interval, frame rate, and quality of the resulting file. This functionality is especially useful for tracking the transient evolution of the circumplanetary flow and for the visual inspection of the stability and convergence of the simulations. The FARGOpy graphical interface provides a useful tool both for quality control and validation of the simulations and for the exploratory analysis of the three-dimensional gas dynamics, significantly reducing

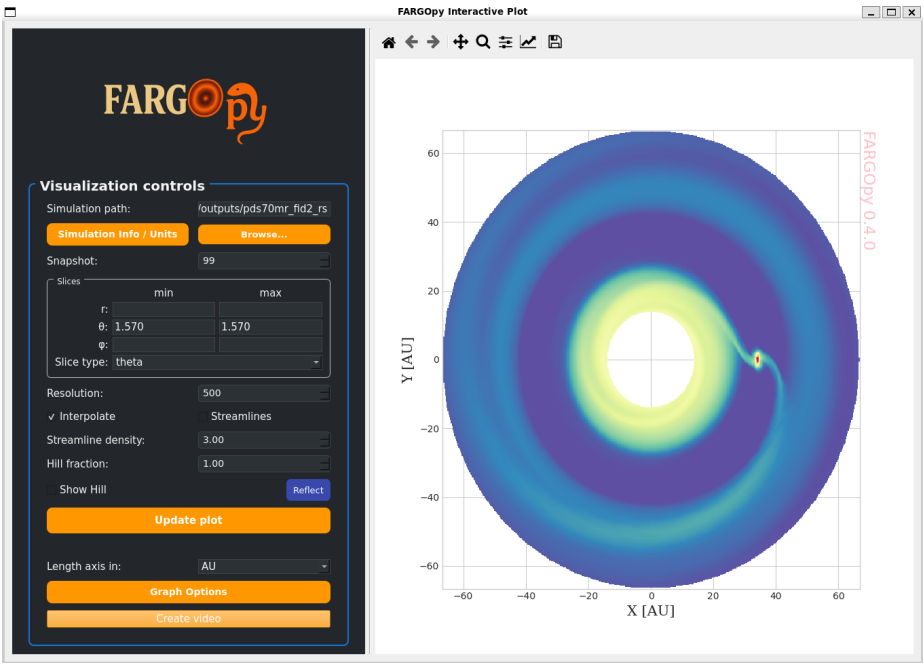


Figure 5: Interactive graphical interface of FARGOpy used for the exploration, control, and visualization of data produced by hydrodynamic simulations with FARGO3D. On the left side are the controls for selecting the simulation, time navigation, definition of spatial cuts in spherical coordinates, interpolation parameters, and visualization options. The right panel presents the corresponding visualization of the selected field, generated dynamically from the simulation data, with support for unit changes, superposition of streamlines, and visual diagnostics of the circumplanetary flow.

post-processing time and improving the reproducibility and traceability of the visual analysis.

4.5. Package availability, installation, compatibility, and prerequisites

The FARGOpy package is publicly distributed as open-source software[1]. The source code, together with the development history, usage examples, and associated documentation, is available in its official repository. Stable versions of the package are also published on the Python Package Index (PyPI)[2], which enables direct installation within the standard Python ecosystem.

4.5.1. Download and installation

The recommended way to download and install FARGOpy is via the `pip` package manager, which ensures a reproducible installation and automatic dependency resolution (`pip install fargopy`). Alternatively, for development purposes or to access the most recent version of the code, the package can be obtained directly from the GitHub repository and installed in editable mode, allowing any modifications made to the source code to be reflected immediately in the working environment.

[1] Official FARGOpy repository: `https://github.com/seap-udea/fargopy.git`.

[2] FARGOpy on PyPI: `https://pypi.org/project/fargopy/`.

4.5.2. Compatibility and use in collaborative environments

FARGOpy can be used on the main operating systems commonly employed in scientific work. In particular, it is compatible with Linux and macOS for the full set of functionalities. On Windows, compatibility is limited: post-processing, analysis, and data interpolation tools are available, but features associated with control, execution, and direct automation of simulations with FARGO3D from within FARGOpy are not fully supported. Consequently, the use of FARGOpy under Windows is primarily geared toward the analysis and visualization of previously generated results.

The package can also be used in cloud-based collaborative development environments such as Google Colab. This compatibility facilitates the execution of post-processing workflows, the reproducible analysis of simulations, and the sharing of notebooks without requiring a local installation. However, it should be noted that some functionalities associated with advanced visualization or three-dimensional rendering may be subject to the inherent limitations of the execution environment.

Currently, FARGOpy is compatible exclusively with versions of `Python>=3.10`. The use of other Python versions is not guaranteed and may lead to unsupported behavior.

4.5.3. Prerequisites

FARGOpy is built on a set of libraries from the Python scientific ecosystem that enable the systematic post-processing of three-dimensional hydrodynamical simulations, including the reconstruction of continuous fields from discrete

meshes, the quantitative analysis of physical quantities, and the visualization of two- and three-dimensional structures. The core of the package is based on `numpy` and `scipy`, which provide the fundamental tools for efficient manipulation of multidimensional arrays, evaluation of numerical operators, and interpolation of hydrodynamical variables on arbitrary geometries, all of which are central aspects for the analysis of circumplanetary disks.

The structured management of data and the statistical analysis of derived quantities, such as radial profiles, accretion rates, and time series, are carried out using `pandas`, which facilitates the reproducible organization of results obtained from multiple simulation snapshots. Scientific visualization relies primarily on `matplotlib`, used for generating publication-quality two-dimensional figures, while `plotly` and `paraview` are employed for interactive exploration and three-dimensional rendering of scalar and vector fields, including density isosurfaces and streamlines.

Interactive exploration of the data and reproducible execution of the analyses are performed in Jupyter-like environments using `ipython`, `ipywidgets`, `ipympl`, and `nbformat`, enabling dynamic inspection of hydrodynamical fields and fine-grained control of post-processing parameters. To improve computational efficiency when analyzing large data volumes, `joblib` and `psutil` are used for task parallelization and system resource management, respectively.

Finally, code validation and verification of the correct implementation of the different `FARGOpy` modules are carried out using `pytest`. Some specific functionalities of the package can optionally rely on `PyQt5` for building graphical interfaces, as well as on `scikit-learn` for specific applications of exploratory analysis and data classification. Although not all capabilities of the package require all of these dependencies simultaneously, their inclusion ensures full compatibility with the set of post-processing tools developed and used in this research.

Acknowledgments

The authors thank the *FONDECYT Regular No. 1241818* project for providing access to high-performance computing resources. This work was made possible through the use of open-source software developed by the scientific community. In particular, the use of the `FARGO3D` hydrodynamic code (Benítez-Llambay and Masset, 2016). Data analysis and visualization were carried out using tools from the Python ecosystem, including `Numpy` (Harris et al., 2020), `Scipy` (Virtanen et al., 2020b), `Matplotlib` (Hunter, 2007), `Pandas` (McKinney, 2010) and ParaView (Ahrens et al., 2005), The authors would like to thank Project `Jupyter` (Kluyver et al., 2016), `Google Colab` (Google, 2024; Bisong, 2019), and `GitHub` (GitHub, Inc., 2024) for the computational and collaboration tools used in this study.

References

Ahrens, J., Geveci, B., Law, C., 2005. Paraview: An end-user tool for large data visualization, in: Visualization Handbook. Elsevier.

Benítez-Llambay, P., Masset, F.S., 2016. FARGO3D: A New GPU-oriented Hydrodynamical Code. ApJS 223, 11. doi:10.3847/0067-0049/223/1/11.

Bisong, E., 2019. Google Colaboratory. Apress, Berkeley, CA. pp. 59–64. doi:10.1007/978-1-4842-4470-8_7.

GitHub, Inc., 2024. Github. https://github.com/. Accessed: 2024-05-20.

Google, 2024. Google colaboratory. https://colab.research.google.com/. Accessed: 2024-05-20.

Harris, C.R., Millman, K.J., van der Walt, S.J., et al., 2020. Array programming with NumPy. Nature 585, 357–362. doi:10.1038/s41586-020-2649-2.

Hunter, J.D., 2007. Matplotlib: A 2d graphics environment. Computing in Science & Engineering 9, 90–95. doi:10.1109/MCSE.2007.55.

Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., et al., 2016. Jupyter notebooks – a publishing format for reproducible computational workflows, in: Positioning and Power in Academic Publishing: Players, Agents and Agendas, IOS Press. pp. 87–90. URL: https://pub.uni-bielefeld.de/record/2948545.

McKinney, W., 2010. Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference , 56–61.

Virtanen, P., Gommers, R., Oliphant, T.E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S.J., Brett, M., Wilson, J., Millman, K.J., Mayorov, N., Nelson, A.R.J., Jones, E., Kern, R., Larson, E., Carey, C.J., Polat, İ., Feng, Y., Moore, E.W., VanderPlas, J., Laxalde, D., Perktold, J., Cimrman, R., Henriksen, I., Quintero, E.A., Harris, C.R., Archibald, A.M., Ribeiro, A.H., Pedregosa, F., van Mulbregt, P., SciPy 1.0 Contributors, 2020a. Scipy 1.0: Fundamental algorithms for scientific computing in python. Nature Methods 17, 261–272. doi:10.1038/s41592-019-0686-2.

Virtanen, P., Gommers, R., Oliphant, T.E., et al., 2020b. SciPy 1.0: Fundamental algorithms for scientific computing in Python. Nature Methods 17, 261–272. doi:10.1038/s41592-019-0686-2.